

Lecture 2: Data Representation I — Bits, Bytes & Integers

Session 2 · Mon Aug 31, 2026

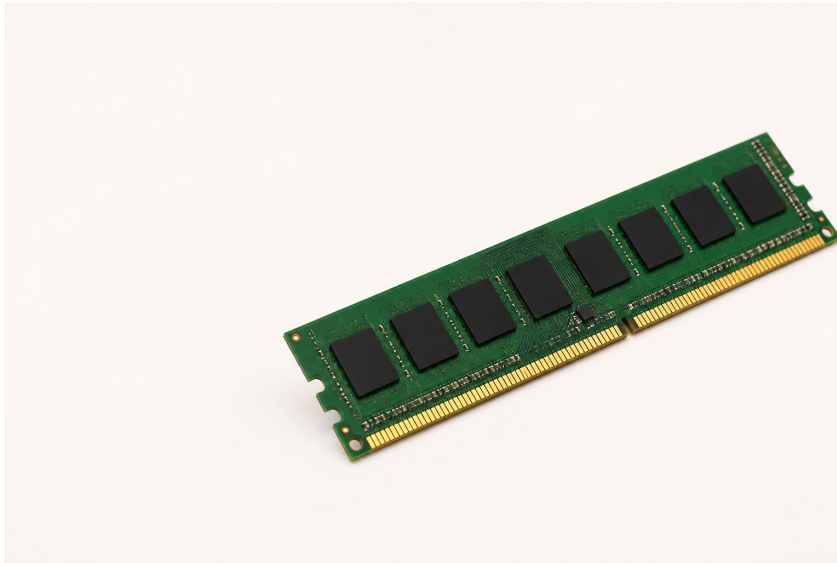
Contents

From Physical Memory to One Byte	1
One Byte, Two Readings	2
Counting in Binary	2
Counting Up	3
Writing the Same Number Three Ways	3
Why Hex Fits Bytes So Well	3
Characters Are Just Numbers	4
The Same Ideas in Python	4
Bit-Level Operations	5
Bitwise Operations, One Column at a Time	5
De Morgan's Laws	5
Shifting Bits	6
Test, Set, Clear, Toggle	7
What Two's Complement Means	7
The Range It Buys You	7
Sign Extension	8
Truncation	8
Size of Integer Types	8
Byte Ordering (Endianness)	9
AI Systems Connection	9
Practice	9
Reading	10

From Physical Memory to One Byte

That is a **RAM module** — physical hardware, many chips, not one byte.

Memory is **byte-addressable**: every address names one **8-bit byte**, and a byte is the smallest unit a C program can address on its own.

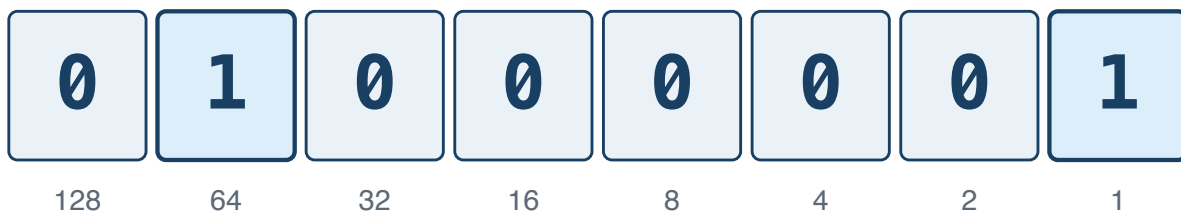


One byte = 8 bits. Today is about what fits in one, and how to work on it.

One Byte, Two Readings

Start with one byte: eight switches that are each either 0 or 1.

One byte: the bit pattern for 65



$$64 + 1 = 65$$

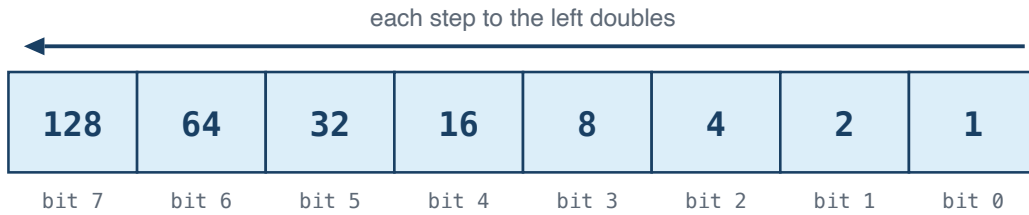
as text (ASCII): A

The computer stores only the bits; the program decides how to read them.

Counting in Binary

Why does **01000001** mean 65? Because each place is worth twice the one to its right — the same idea as decimal's ones, tens and hundreds, with 2 instead of 10.

Each place is worth twice the place to its right



Turn a bit on and you add its place value.

$$128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$$

Counting Up

Counting works exactly the way it does in decimal — when a column runs out of digits, it rolls over and carries into the next one:

0 = 0000	3 = 0011	6 = 0110	9 = 1001
1 = 0001	4 = 0100	7 = 0111	10 = 1010
2 = 0010	5 = 0101	8 = 1000	11 = 1011

Each extra bit doubles the patterns available: 8 bits give $2^8 = 256$. So an unsigned byte runs 0–255, and an N-bit unsigned value runs 0 to $2^N - 1$ — its **UMax**, modulo 2^N .

Writing the Same Number Three Ways

A number does not change when you write it differently. C lets you say which notation you are using with a **prefix**:

Written	Prefix means	Value
42	no prefix — plain decimal	forty-two
0b101010	0b — read the digits as binary	forty-two
0x2A	0x — read the digits as hexadecimal	forty-two

All three are the same value to the compiler. You pick whichever makes the thing you are doing readable: decimal for counting, binary for seeing bits, hex for writing bytes compactly.

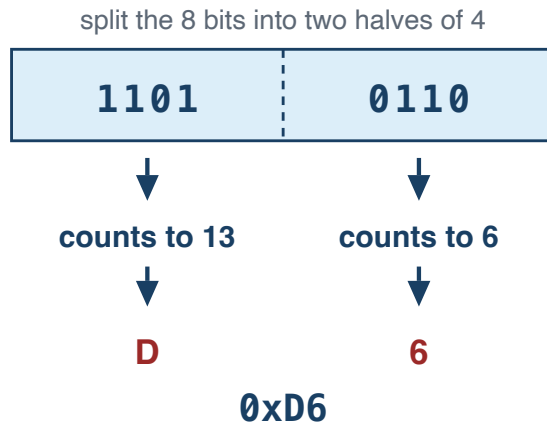
Note

0b is not standard until C23. GCC and Clang accept it, and these slides use it to show bit patterns, but in ISO C17 write **0x2A** or **42**.

Why Hex Fits Bytes So Well

Eight ones and zeros is easy to miscount. Four bits count 0–15 — exactly one hex digit — so one byte is always two digits, and you can still read the bits straight off the page.

One byte = two hex digits



**4 bits count 0 to 15,
so we need 6 extra digits:**

10 = A
11 = B
12 = C
13 = D
14 = E
15 = F

0 through 9 stay as they are.

Characters Are Just Numbers

The rule that turns 65 into A is **ASCII** — the American Standard Code for Information Interchange — a table agreed in the 1960s that gives every character a number from 0 to 127.

Characters	Codes	Worth knowing
'0'–'9'	48–57 (0x30–0x39)	consecutive, so <code>c - '0'</code> gives the digit
'A'–'Z'	65–90 (0x41–0x5A)	consecutive
'a'–'z'	97–122 (0x61–0x7A)	exactly 32 more than the capitals

```
char c = '7';
int d = c - '0';           // 7
int is_digit = (c >= '0' && c <= '9'); // 1
```

In C the literal 'A' is the number 65. Nothing clever is happening.

The Same Ideas in Python

C

```
int x = 65;
printf("%d\n", x);    // 65
printf("%c\n", x);    // A

unsigned a = 12, b = 10;
printf("%u\n", a & b); // 8
printf("%u\n", a << 2); // 48
```

Python

```
x = 65
print(x)        # 65
print(chr(x))   # A

a, b = 12, 10
print(a & b)     # 8
print(a << 2)   # 48
```

The operators are the same. The difference is the box: a C value has a fixed width, and a Python integer just grows.

Bit-Level Operations

For these bit patterns, C provides six bitwise operators:

Op	Name	Example
&	AND	<code>0b1100 & 0b1010 = 0b1000</code>
	OR	<code>0b1100 0b1010 = 0b1110</code>
^	XOR	<code>0b1100 ^ 0b1010 = 0b0110</code>
~	NOT	<code>~0b1100 = 0b1111...11110011</code> (every bit flips, including the leading zeros)
<<	Left shift	<code>0b0011 << 2 = 0b1100</code>
>>	Right shift	<code>0b1100 >> 2 = 0b0011</code>

Warning

Right shift of signed integers is **implementation-defined** in C. GCC uses **arithmetic** right shift (preserves sign bit). Don't rely on this unless you're sure of your compiler.

Bitwise Operations, One Column at a Time

The same two inputs, three separate rules

Every output bit is decided on its own, from just the two bits directly above it.

INPUTS					
a	1	1	0	0	= 1100
b	1	0	1	0	= 1010
a & b	1	0	0	0	AND = 1000 1 only where both are 1
a b	1	1	1	0	OR = 1110 1 where either is 1
a ^ b	0	1	1	0	XOR = 0110 1 where they differ

De Morgan's Laws

Two rules let you rewrite any mix of &, | and ~ — and they are how Lab 1 asks you to build ^ out of nothing but ~ and &.

Flip the whole thing, and AND becomes OR

$$\sim(x \ \& \ y) == \sim x \ | \ \sim y$$

$$\sim(x \ | \ y) == \sim x \ \& \ \sim y$$

x	y	x & y	$\sim(x \ \& \ y)$	$\sim x \ \ \sim y$
0	0	0	1	1
0	1	0	1	1
1	0	0	1	1
1	1	1	0	0

same in all four rows

Two inputs give four possible rows, so checking all four is a complete proof.

Shifting Bits

Shifting slides every bit sideways

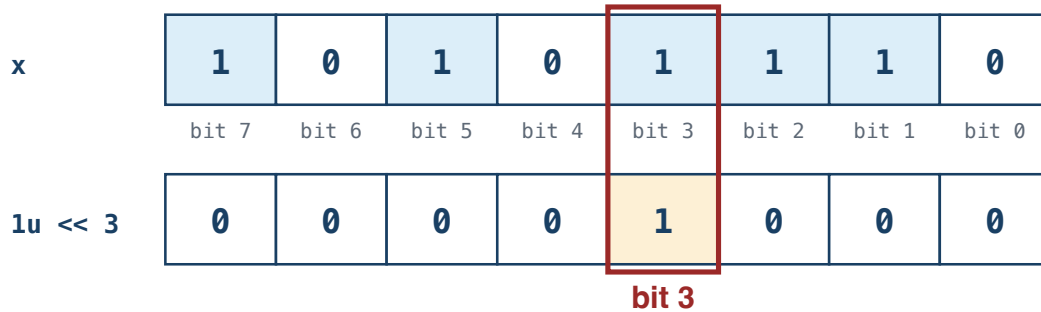
x	0	0	0	0	1	1	0	0	= 12
	←								
x << 2	0	0	1	1	0	0	0	0	= 48 (× 4)
	→								
x >> 2	0	0	0	0	0	0	1	1	= 3 (÷ 4)

Bits pushed off the end are gone for good; zeros come in behind them.

Shifting left by n multiplies by 2^n . Shifting an unsigned value right by n divides by 2^n .

Test, Set, Clear, Toggle

$1u \ll 3$ builds a mask with exactly one bit set



Bit 0 is the rightmost — the ones place. The mask is 0 everywhere except the bit you want.

```

unsigned x = 0xAE;           // 10101110
x |= (1u << 3);             // SET   - bit 3 becomes 1
x &= ~(1u << 3);            // CLEAR - bit 3 becomes 0
x ^= (1u << 3);             // TOGGLE - bit 3 flips
if (x & (1u << 3)) { }     // TEST  - nonzero when bit 3 is 1

```

Those four are most of Lab 1. Keep the value **unsigned** — shifting into a sign bit is undefined.

What Two's Complement Means

Complement means the part that completes a whole. Flip every bit of x to get its *one's complement*; add 1 to that and you have its **two's complement**, which is how C writes $-x$:

To write -5 in 8 bits

1. Start with +5

00000101

↓ flip every 0 ↔ 1

2. One's complement

11111010

+ 1

↓

3. Two's complement

11111011

= -5

More 8-bit examples

+1: 00000001

-1: 11111111

0: 00000000

two's: 00000000

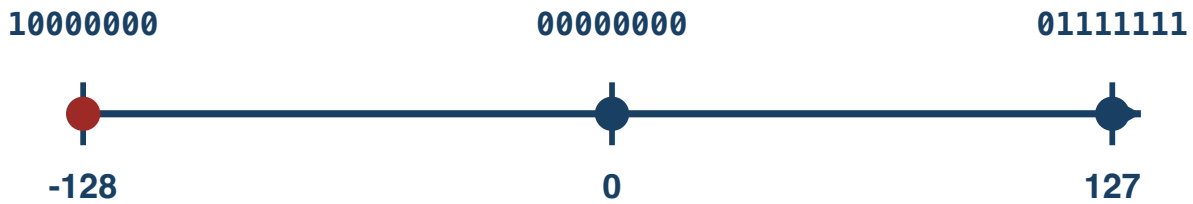
Zero stays zero.

A ninth bit is simply discarded — 8 bits keep only their rightmost 8.

The Range It Buys You

An N -bit two's complement covers $-2^{(N-1)}$ to $2^{(N-1)} - 1$:

8-bit two's-complement values



There is one more negative value than positive value.

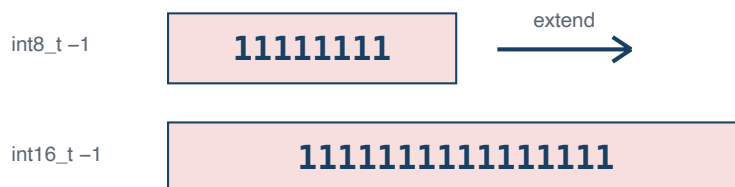
$T_{\min} = -T_{\max} - 1$ — there is no positive 128 in 8 bits, so negating $T_{\min}$ overflows and is undefined.

Sign Extension

When converting a smaller signed type to a larger one, copy the sign bit:

```
int8_t  x = -1;    // 0b11111111
int16_t y = x;     // 0b1111111111111111 = -1 ✓
```

Copy the sign bit into the new high bits

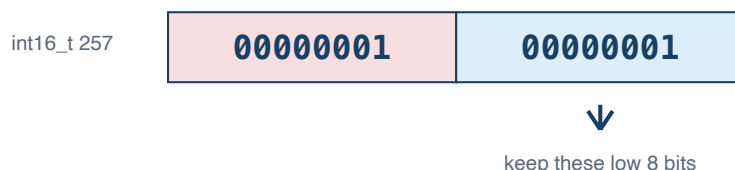


Truncation

When converting larger to smaller, drop the high bits:

```
int16_t x = 257;
uint8_t y = x;    // 1: conversion to unsigned is modulo 256
```

Keep the low bits; discard the high bits



`int8_t result = 00000001 = 1`

Size of Integer Types

```
printf("char:      %zu\n", sizeof(char));    // 1 C byte (not always 8 bits)
printf("short:     %zu\n", sizeof(short));    // 2
printf("int:       %zu\n", sizeof(int));      // 4
```

```
printf("long:      %zu\n", sizeof(long));    // 8 (LP64)
printf("long long: %zu\n", sizeof(long long)); // 8
printf("pointer:   %zu\n", sizeof(void*));   // 8
```

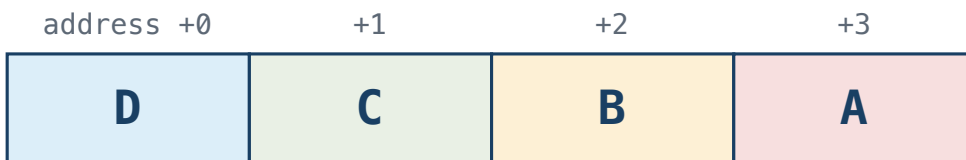
Tip

Use `sizeof` and fixed-width types (`int32_t`, `uint64_t`) when exact sizes matter. Never assume `int` is 4 bytes; use `CHAR_BIT` when bit width matters. Plain `char` is its own trap: whether it is signed is implementation-defined, so say `signed char` or `unsigned char` when it matters.

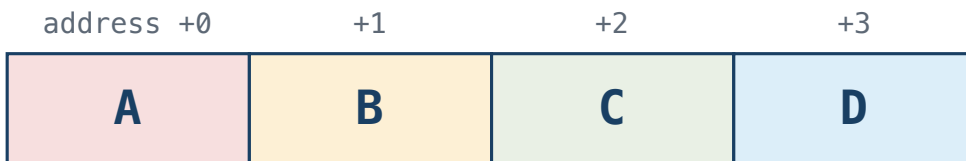
Byte Ordering (Endianness)

An `int` occupies 4 consecutive bytes, so those 4 bytes have to be laid out in some order. **Endianness** is that order. One question matters: **which byte goes at the lowest address?**

Little-endian: D is first in memory



Big-endian: A is first in memory



```
int x = 1;    // ask the machine: which byte of x sits at the lowest address?
if (*(unsigned char *)&x == 1) printf("Little-endian\n");
else                             printf("Big-endian\n");
```

AI Systems Connection

Quantization is one important reason large models can fit on more hardware. A common symmetric `int8` scheme represents a block of weights with 8-bit signed values and a scale factor:

$$\text{real_value} \approx \text{scale} \times \text{int8_value} \quad \text{scale} = \text{max_abs} / 127$$

Everything today shows up here. Two `int8` values need 16 bits for their product, so a long dot product accumulates into something wider — often `int32_t`. And if a signed weight is loaded as *unsigned* into a wider register, sign extension does not happen, negatives become huge positives, and the answer is quietly wrong.

Try it: quantize a float array with `scale = max_abs / 127`, dequantize, and measure the error.

Practice

- Write `0xD6` in binary, and `10110010` in hex, without using a computer.
- What does `x & 1` tell you about `x`? And what does `x & (x - 1)` do? Try both on `0b0110` by hand.
- Write one expression that clears bit 5 of `unsigned x`, and one that tests whether bit 0 and bit 7 are both set.
- What is `TMax` for a 32-bit `int`? What is `TMin`?
- Write a C expression that checks if your machine is little-endian.
- Is left-shifting a 32-bit `int` by 32 positions defined in C? Why not?
- Why is `-TMin` not representable in the same signed type?

Reading

- CS:APP §2.1 (Information Storage)
- CS:APP §2.2 (Integer Representations)